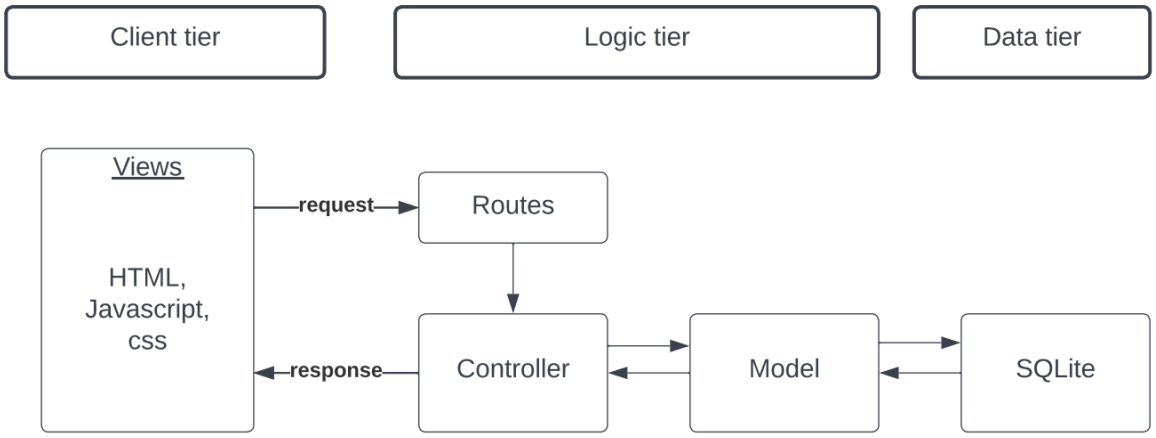
1. Architecture Overview:

To provide a high-level understanding of the web application's architecture, I've created a schematic diagram using Lucid Chart. The diagram illustrates the three-tier architecture involving the client, server, and database.



Client Tier: Represents the user interface and interaction components.

Server Tier: Depicts the Express.js server responsible for handling requests and responses.

Database Tier: Represents SQLite, the database used for storing user data, articles, drafts, and comments.

2. Extension Description:

2.1 Rich Text Editor Implementation:

A standout feature in the web application is the implementation of a rich text editor using the Quill.js library. This editor, seamlessly integrated into the project by including essential script files, empowers users to create and edit articles and drafts with a user-friendly interface. The initiation of the Quill editor instance is detailed, specifying the target element where the editor should be rendered, and further customization of the toolbar options to cater to specific user needs.

Relevant Code Snippet and Implementation Steps:

Integration: Incorporated the Quill.js library into the project by including the necessary script files.

```
<link href="https://cdn.quilljs.com/1.3.6/quill.snow.css" rel="stylesheet">
```

Initialization: Created an instance of the Quill editor, specifying the target element where the editor should be rendered.

```
const quill = new Quill('#editor', {
      theme: 'snow',
      modules: {
        toolbar: [
          // Customize toolbar options as needed
          ['bold', 'italic', 'underline', 'strike'],
          [{ 'list': 'ordered' }, { 'list': 'bullet' }],
          ['link', 'image'],
          ['clean']
        ]
      }
    });
```

Content Handling: Implemented logic to handle the content generated by the editor, ensuring proper storage and retrieval in the database.

```
commentForm.addEventListener('submit', (event) => {
      const currentArticleId = articleIds[0];
      articleIdInput.value = currentArticleId;

      // Set the value of the hidden input with the HTML content from Quill
      const commentHtml = quill.root.innerHTML;
      document.getElementById('comment').value = commentHtml;
    });
```

Code snippets extracted from the article.ejs views reveal the intricacies of Quill editor initialization and content handling. The logic implemented ensures proper storage and retrieval of content in the database, highlighting the meticulous attention given to data management.

2.2 Inline CSS Styling:

In addition to the rich text editor, I implemented the ability for users to apply inline CSS styles to their articles. This provides a more personalized and customizable approach to article formatting.

Implementation Details:

User Interface: I added CSS styles for all the pages with simple colors and design to make them look relaxing for the users eyes.Also, added UI elements for users to apply inline styles such as bold, italic, underline, and text color.

Relevant Code Snippet:

It’s in all the ejs in views, where the inline.

3. Summary and Conclusion:

The implemented web application architecture follows a robust three-tier structure, ensuring efficient handling of user interactions, server-side logic, and data storage. The Quill.js rich text editor enhances the user experience, providing a powerful and intuitive tool for content creation. The integration of inline CSS styling further empowers users to customize their articles according to their preferences.

The provided diagram and code snippets offer a glimpse into the project's structure and key functionalities. This web application lays a solid foundation for future enhancements and scalability.